

MODULE 1 – DESIGN PATTERNS AND PRINCIPLES

PRINCIPLES:

1. Single Responsibility Principle (SRP)

A class should have only one responsibility.

Example:

Python code

```
class Employee:
```

```
    def calculate_salary(self):
```

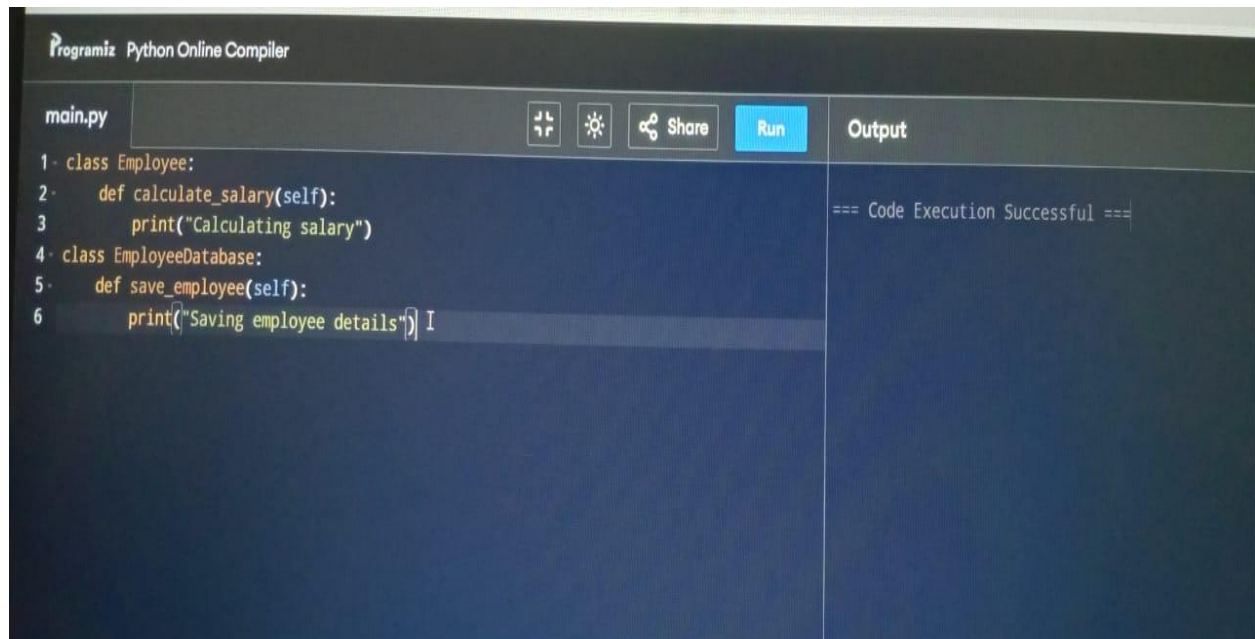
```
        print ("Calculating salary")
```

```
class Employee Database:
```

```
    def save_employee(self):
```

```
        print ("Saving employee details")
```

My python compiler which I run the code:



```
main.py
1 class Employee:
2     def calculate_salary(self):
3         print("Calculating salary")
4 class EmployeeDatabase:
5     def save_employee(self):
6         print("Saving employee details") I

Run

Output
=== Code Execution Successful ===
```

2. Open Closed Principle (OCP)

Software entities should be open for extension but closed for modification.

Example:

Python code:

```
class Shape: def area(self): pass
```

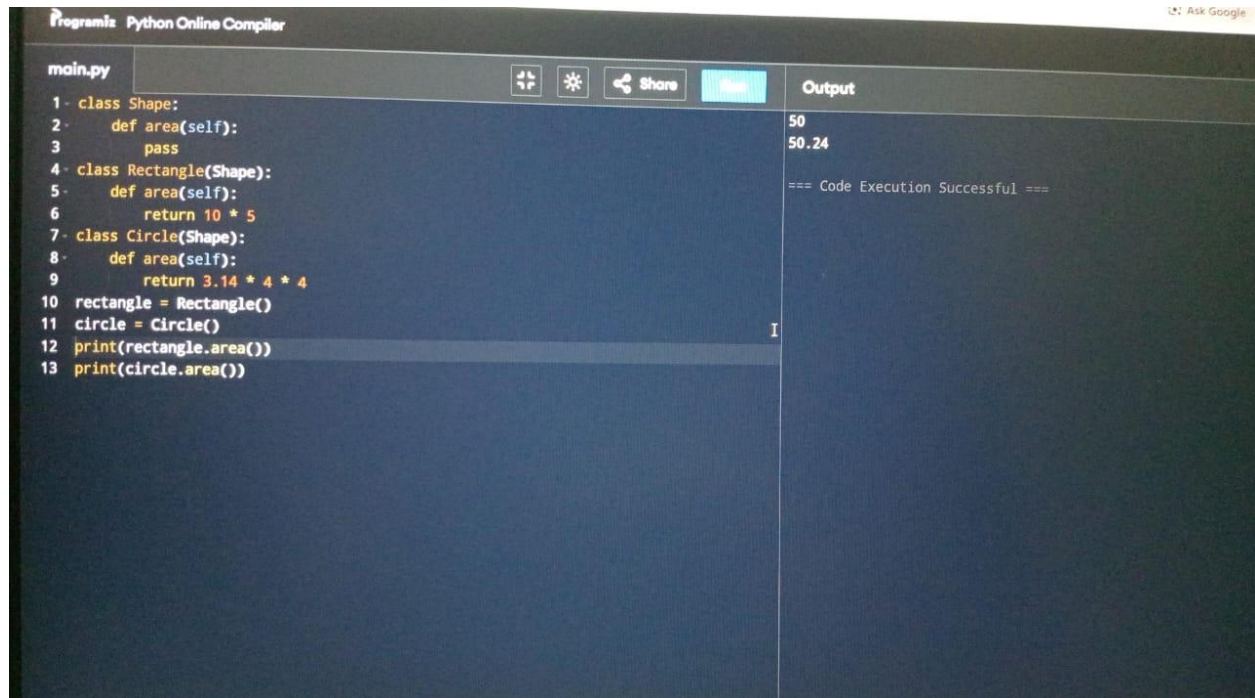
```
class Rectangle (Shape): def area(self): return 10 * 5
```

```
class Circle (Shape): def area(self): return 3.14 * 4 * 4
```

```
rectangle = Rectangle () circle = Circle ()
```

```
print (rectangle. Area ()) print (circle. Area ())
```

My practise screenshot:



The screenshot shows a Python Online Compiler interface. The code in the editor is as follows:

```
1 class Shape:
2     def area(self):
3         pass
4 class Rectangle(Shape):
5     def area(self):
6         return 10 * 5
7 class Circle(Shape):
8     def area(self):
9         return 3.14 * 4 * 4
10 rectangle = Rectangle()
11 circle = Circle()
12 print(rectangle.area())
13 print(circle.area())
```

The output on the right shows the results of the execution:

```
50
50.24
=== Code Execution Successful ===
```

3. Liskov Substitution Principle (LSP)

A derived class should be able to replace its base class without affecting the correctness of the program.

Example:

```
class Bird:
```

```
    def move(self):
```

```
        pass
```

```
class Sparrow (Bird):
```

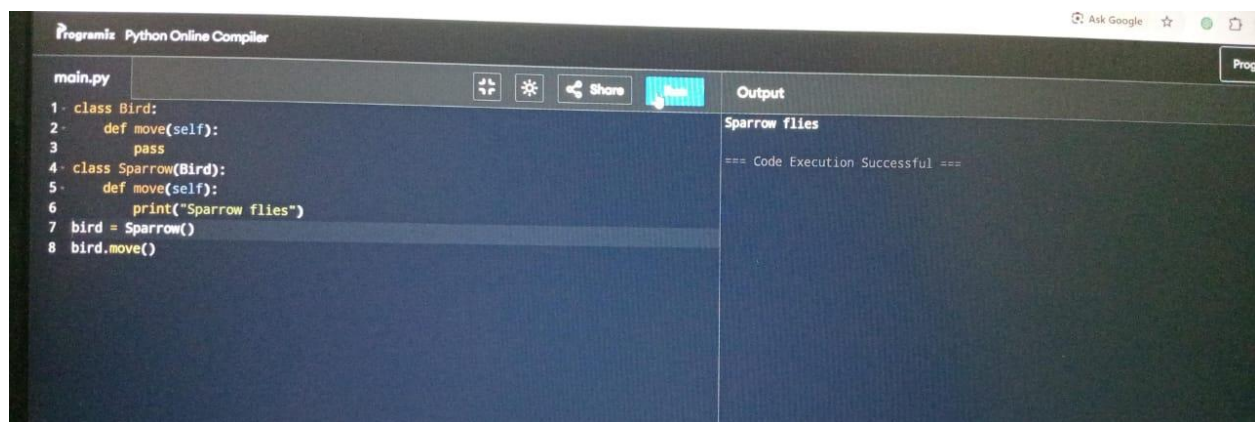
```
    def move(self):
```

```
print ("Sparrow flies")
```

```
bird = Sparrow ()
```

```
bird. move()
```

My screenshot:

A screenshot of a web-based Python IDE. The editor on the left shows a file named 'main.py' with the following Python code:

```
1 class Bird:
2     def move(self):
3         pass
4 class Sparrow(Bird):
5     def move(self):
6         print("Sparrow flies")
7 bird = Sparrow()
8 bird.move()
```

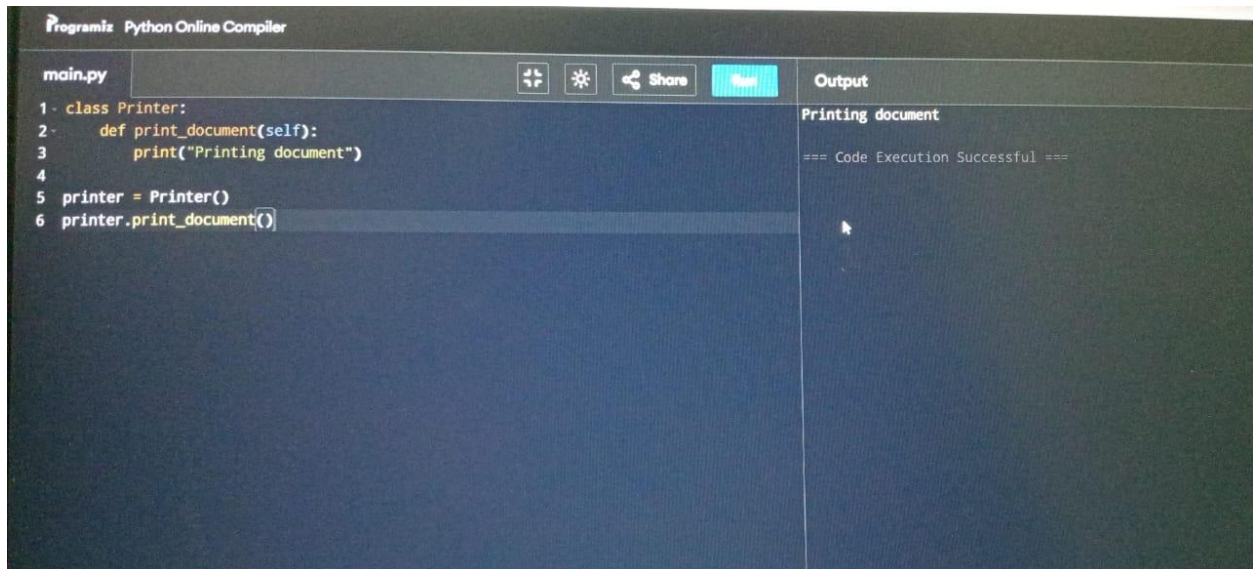
The 'Run' button is highlighted. The output panel on the right shows the result of the execution:

```
Sparrow flies
=== Code Execution Successful ===
```

4. Interface Segregation Principle (ISP)

Classes should not be forced to implement interfaces they do not use.

My code:



The screenshot shows a web-based Python compiler interface. The title bar reads "Programiz Python Online Compiler". The code editor on the left contains the following Python code:

```
main.py
1 class Printer:
2     def print_document(self):
3         print("Printing document")
4
5 printer = Printer()
6 printer.print_document()
```

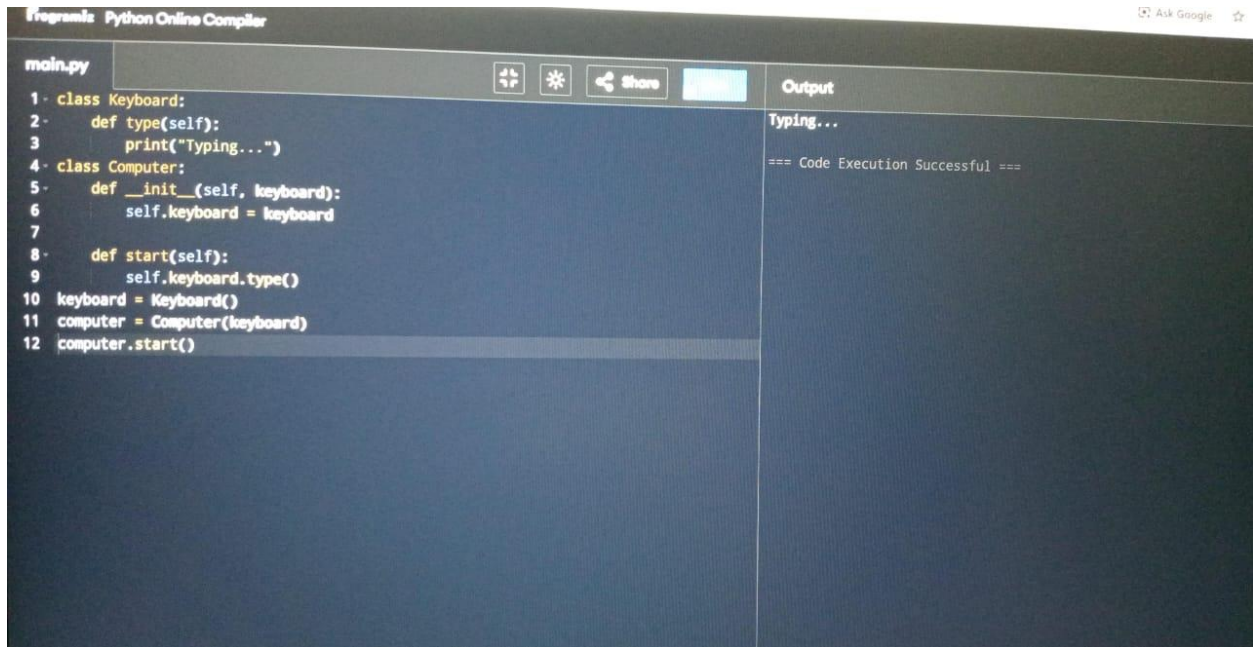
On the right side, the "Output" panel displays the execution results:

```
Printing document
=== Code Execution Successful ===
```

5. Dependency Inversion Principle (DIP)

High-level modules should depend on abstractions instead of concrete implementations.

My screenshot:

The image shows a screenshot of a web-based Python compiler interface. The top bar includes the text "Programiz Python Online Compiler" and a search icon with the text "Ask Google". Below the bar, the code editor is titled "main.py" and contains the following Python code:

```
1 class Keyboard:
2     def type(self):
3         print("Typing...")
4 class Computer:
5     def __init__(self, keyboard):
6         self.keyboard = keyboard
7
8     def start(self):
9         self.keyboard.type()
10 keyboard = Keyboard()
11 computer = Computer(keyboard)
12 computer.start()
```

To the right of the code editor is an "Output" panel. It displays the text "Typing..." followed by "=== Code Execution Successful ===". The interface also features icons for a full-screen view, a sun icon, and a "Share" button.

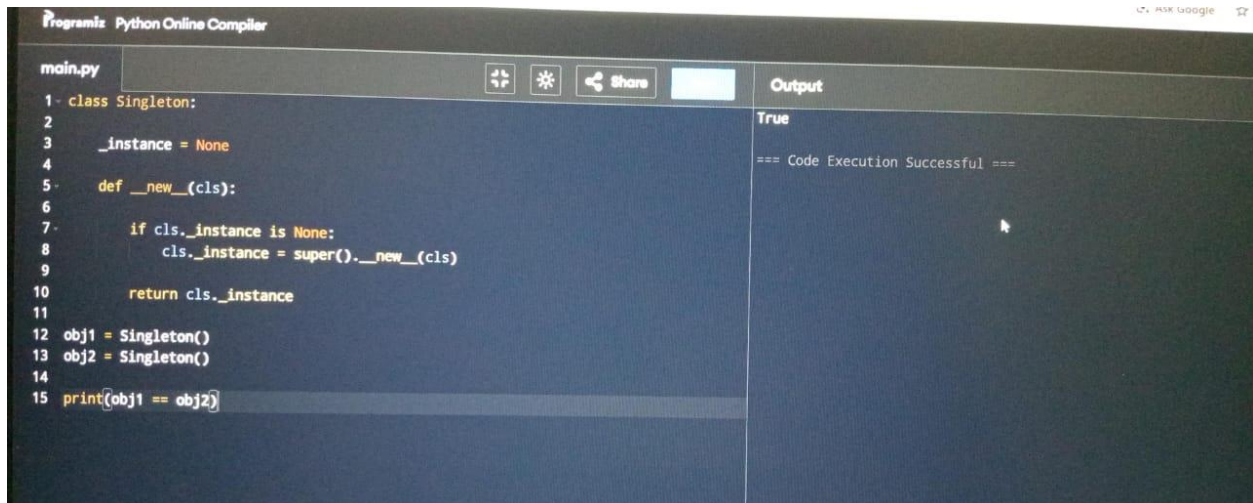
Design Patterns:

Design patterns are reusable solutions to common software design problems.

1. Singleton Pattern

Ensures that only one object of a class is created.

Example:



The screenshot shows a web-based Python IDE. The editor on the left contains a Python script for a Singleton class. The output panel on the right shows the result of the execution.

```
main.py
1 class Singleton:
2
3     _instance = None
4
5     def __new__(cls):
6
7         if cls._instance is None:
8             cls._instance = super().__new__(cls)
9
10        return cls._instance
11
12 obj1 = Singleton()
13 obj2 = Singleton()
14
15 print(obj1 == obj2)
```

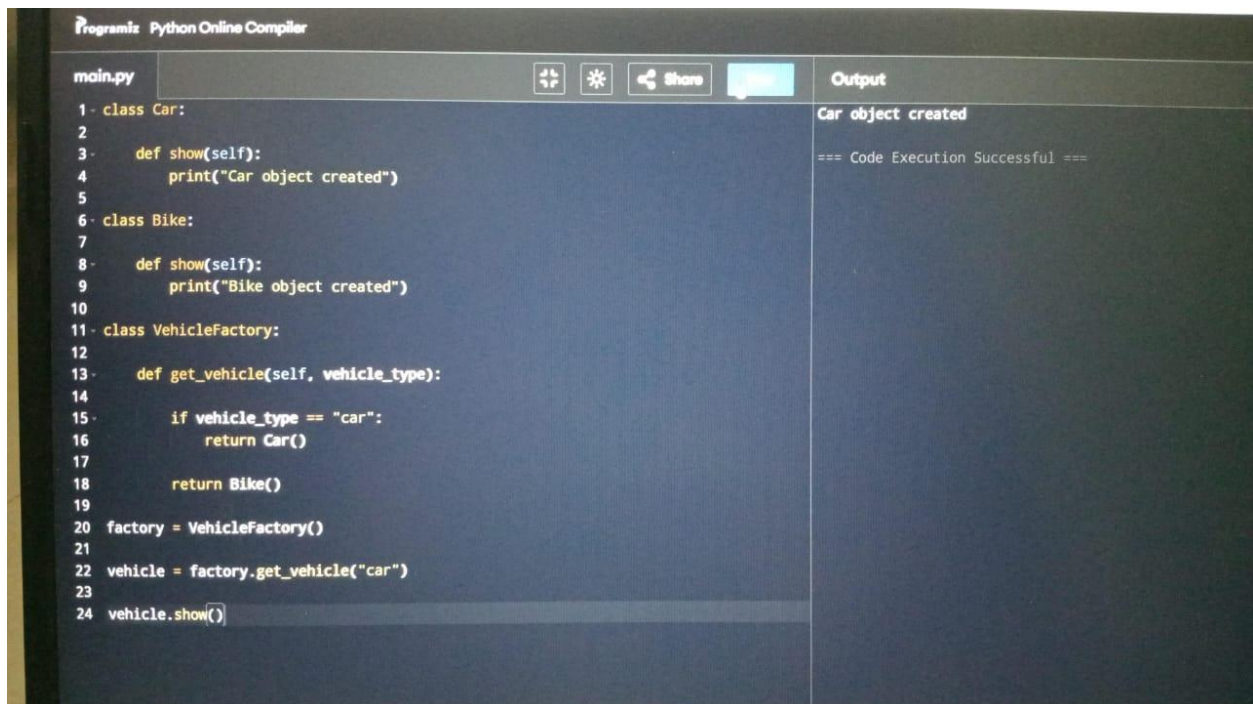
Output

```
True
=== Code Execution Successful ===
```

2. Factory Pattern:

Creates objects without exposing object creation details.

My Example:



The screenshot shows a web-based Python IDE. The editor on the left contains a Python script for a Factory class. The output panel on the right shows the result of the execution.

```
main.py
1 class Car:
2
3     def show(self):
4         print("Car object created")
5
6 class Bike:
7
8     def show(self):
9         print("Bike object created")
10
11 class VehicleFactory:
12
13     def get_vehicle(self, vehicle_type):
14
15         if vehicle_type == "car":
16             return Car()
17
18         return Bike()
19
20 factory = VehicleFactory()
21
22 vehicle = factory.get_vehicle("car")
23
24 vehicle.show()
```

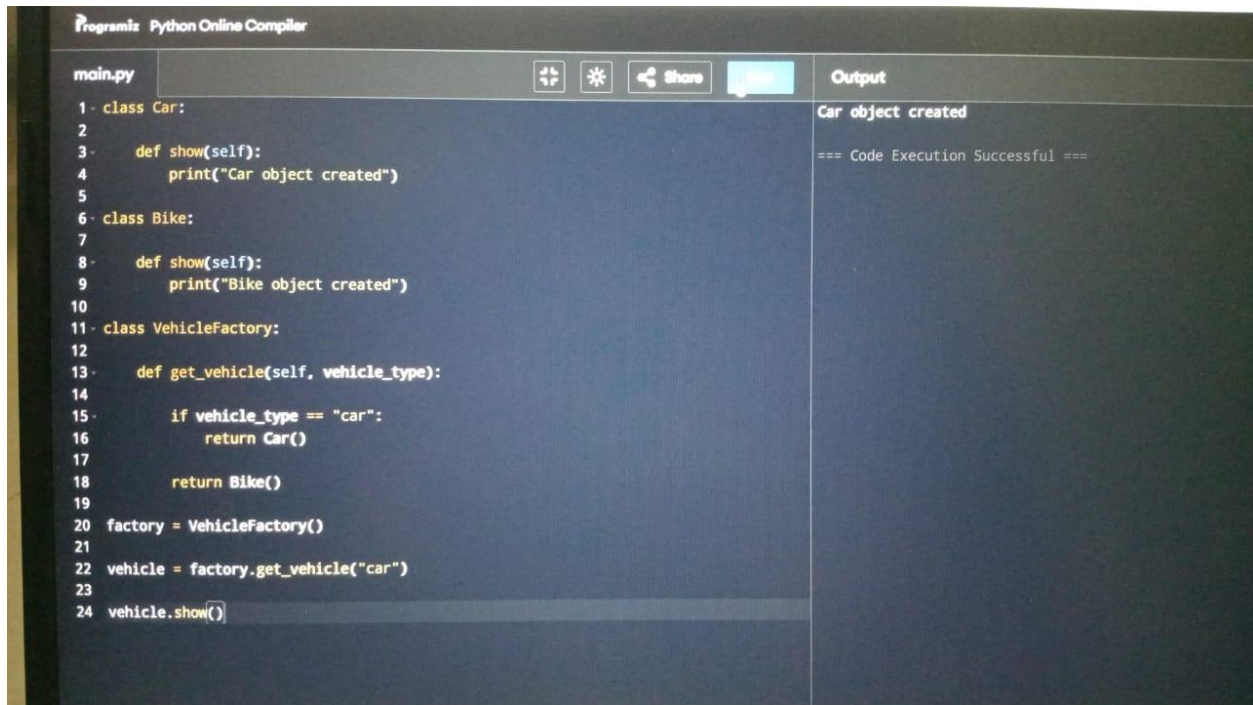
Output

```
Car object created
=== Code Execution Successful ===
```

3. Observer Pattern:

When one object changes, all subscribed objects are notified.

My Example:



The screenshot shows a Python Online Compiler interface. The code in the editor is as follows:

```
1 class Car:
2
3     def show(self):
4         print("Car object created")
5
6 class Bike:
7
8     def show(self):
9         print("Bike object created")
10
11 class VehicleFactory:
12
13     def get_vehicle(self, vehicle_type):
14
15         if vehicle_type == "car":
16             return Car()
17
18         return Bike()
19
20 factory = VehicleFactory()
21
22 vehicle = factory.get_vehicle("car")
23
24 vehicle.show()
```

The output on the right side of the compiler shows:

```
Car object created
=== Code Execution Successful ===
```

4. Strategy Pattern:

Provides multiple ways to perform a task.

My Example:

main.py



Output

```
1 class UPIPayment:
2
3     def pay(self):
4         print("Paid using UPI")
5
6 class CardPayment:
7
8     def pay(self):
9         print("Paid using Credit Card")
10
11 payment = UPIPayment()
12
13 payment.pay()
```

Paid using UPI

=== Code Execution Successful ===